



За пределами GARCH (Часть IV): Реализация анализа разбиений в MQL5

0

Muhammad Minhas Qamar

70

Введение

В первых трех частях серии мы построили полный пайплайн MMAR на Python. Мы загрузили данные EURUSD, подтвердили мультифрактальное масштабирование анализом функции разбиения, оценили показатель Херста и подогнали спектр сингулярностей. Затем мы построили мультипликативный каскад и фракционное броуновское движение, провели моделирование методом Монте-Карло и напрямую сравнили MMAR с GARCH. MMAR показала лучшие результаты. Результаты тестирования подтверждают состоятельность концепции.

Теперь перед нами другая задача — инженерная реализация. Python отлично подходит для исследований и создания прототипов, но не работает внутри MetaTrader 5 нативно. Чтобы MMAR принимала торговые решения в реальном времени, реагировала на каждый тик и без проблем интегрировалась с Тестером стратегий, нужна реализация на языке MQL5. Без внешних зависимостей, скриптов-посредников и задержек из-за межпроцессного взаимодействия. Просто аккуратно организованная библиотека, которую можно подключить к любому советнику и вызывать напрямую.

В этой статье мы приступаем к ее созданию. Мы создадим первый крупный аналитический модуль библиотеки MMAR — механизм анализа разбиений. Этот компонент получает необработанные ценовые данные и вычисляет функцию разбиения при разных временных масштабах и порядках моментов. Затем определяется функция масштабирования $\tau(q)$, оценивается показатель Херста H и проводится проверка, обладают ли данные мультифрактальными свойствами. К концу статьи у нас будет работающий скрипт на языке MQL5: он загрузит исторические бары, проведет анализ и сообщит, являются ли рыночные данные мультифрактальными.

Мы рассмотрим следующие темы:

1. [Архитектура библиотеки и константы](#)
2. [Основа метода наименьших квадратов \(OLS\)](#)
3. [Оценка показателя Херста](#)
4. [Анализ разбиений — основной вычислительный механизм](#)
5. [Запуск анализа на реальных данных](#)
6. [Заключение](#)

Архитектура библиотеки и константы

Прежде чем писать аналитический код, нам нужно договориться о едином наборе обозначений. Библиотека MMAR состоит из взаимосвязанных модулей. Все они используют общие перечисления, структуры и определения типов. Мы помещаем их в общий заголовочный файл MMARConstants.mqh, который подключают остальные модули.

Библиотека построена по многослойной архитектуре. В нижнем слое находятся вспомогательные модули: класс OLS-регрессии и оценщик показателя Херста. Выше расположен механизм анализа разбиений: с помощью обоих модулей он вычисляет функцию масштабирования и оценивает мультифрактальность. В следующих статьях мы добавим модуль подгонки спектра, использующий кривую $\tau(q)$ из анализа разбиений; механизм моделирования, который строит синтетические траектории по выбранному распределению; и модуль прогнозирования методом Монте-Карло, координирующий множество прогонов. На верхнем уровне находится фасадный класс CMMAR, объединяющий компоненты через простой интерфейс подгонки и прогнозирования. Пока будем строить библиотеку снизу вверх.

На рисунке 1 показана эта многослойная архитектура:

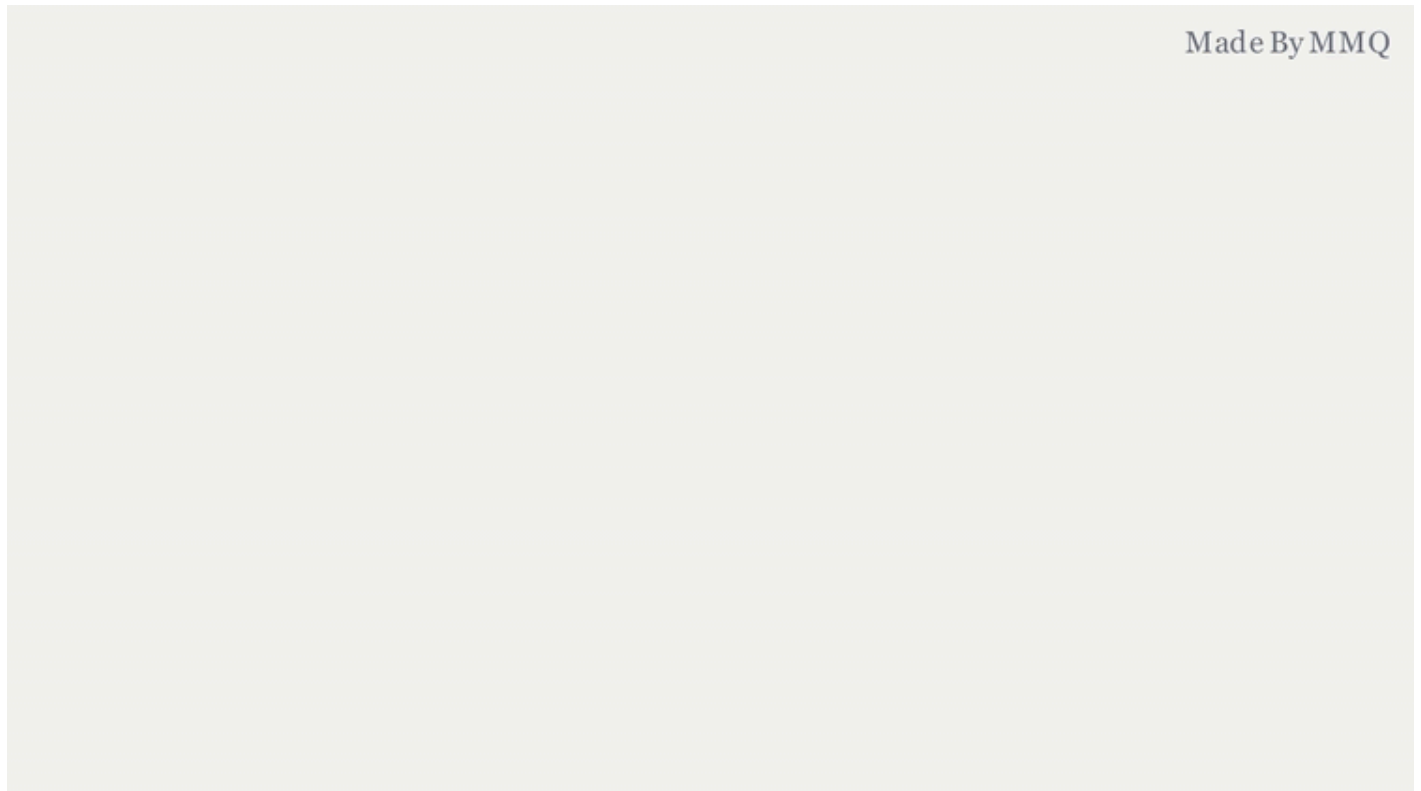


Рис. 1. Архитектура библиотеки MMAR. Золотым цветом выделены три нижних модуля, рассматриваемые в этой статье

Примечание: класс OLS-регрессии и оценщик обобщенного показателя Херста созданы на основе статьи ["Реализация обобщенного показателя Херста и теста коэффициента дисперсии в MQL5"](#). На этой статье основаны наша реализация OLS — автономный контейнер регрессии — и подход к оценке GHE. Мы сократили и изменили оба модуля под задачи пайплайна MMAR, но исходные реализации на языке MQL5, на которых основана наша работа, созданы авторами той статьи.

Начнем с файла констант:

```
//+-----+
//|                                     MMARConstants.mqh |
//|      MMAR Library - Constants, Enumerations, and Data Structures |
//+-----+
#pragma strict

#ifndef MMAR_CONSTANTS_MQH
#define MMAR_CONSTANTS_MQH

//+-----+
//| Cascade multiplier distribution types |
//+-----+
enum ENUM_MMAR_DISTRIBUTION
{
```



```

MMAR_DIST_NORMAL = 0,
MMAR_DIST_BINOMIAL,
MMAR_DIST_POISSON,
MMAR_DIST_GAMMA
};

//+-----+
//| Fractional Brownian Motion synthesis methods |
//+-----+
enum ENUM_MMAR_FBM_METHOD
{
    MMAR_FBM_DAVIES_HARTE = 0,
    MMAR_FBM_CHOLESKY
};

//+-----+
//| Model fitting status |
//+-----+
enum ENUM_MMAR_STATUS
{
    MMAR_STATUS_NOT_FITTED = 0,
    MMAR_STATUS_FITTED,
    MMAR_STATUS_PARTITION_ONLY,
    MMAR_STATUS_ERROR
};

//+-----+
//| Fitted model parameters |
//+-----+
struct MMARParams
{
    double          H;
    double          tau_q[];
    double          q_values[];
    ENUM_MMAR_DISTRIBUTION dist_type;
    double          dist_params[4];
    double          alpha_0;
    double          mean_r_squared;
    double          sample_volatility;
    bool            is_multifractal;
    int             multifractal_score;
};

//+-----+
//| Monte Carlo forecast results |
//+-----+
struct MMARForecast
{
    double          mean_volatility;
    double          std_volatility;
    double          ci_lower;
    double          ci_upper;
    double          median_volatility;
    int             n_simulations;
    int             forecast_horizon;
};

```

```
#endif
//+-----+
```

Перечисление ENUM_MMAR_DISTRIBUTION задает четыре распределения-кандидата для последующей подгонки мультифрактального спектра: нормальное, биномиальное, пуассоновское и гамма-распределение. Они в точности соответствуют четырём теоретическим моделям, рассмотренным нами при реализации на Python. Перечисление ENUM_MMAR_FBM_METHOD задает два метода генерации фракционного броуновского движения: быстрый алгоритм Дэвиса — Харта на основе БПФ и точное, но более медленное разложение Холецкого. Перечисление ENUM_MMAR_STATUS обозначает текущий этап работы экземпляра модели. Это важно при частичном успехе: анализ разбиений может завершиться, а подгонка спектра — дать сбой.

В структуре MMARParams будут храниться полученные из данных показатели Херста, полная кривая функции масштабирования $\tau(q)$, выбранный тип распределения и его параметры, а также результаты диагностики мультифрактальности. Структура MMARForecast хранит результаты моделирования методом Монте-Карло: среднее, медиану, стандартное отклонение и границы доверительного интервала прогнозируемой волатильности. Структуры будут полностью заполняться лишь в следующих частях, но определяем мы их уже сейчас, чтобы у модулей с самого начала был общий интерфейс.

Основа метода наименьших квадратов (OLS)

При анализе функции разбиения нужно подбирать прямые на графиках в логарифмических координатах. Для каждого порядка момента q мы рассчитываем функцию разбиения $S_q(dt)$ при нескольких масштабах dt , логарифмируем $S_q(dt)$ и dt , а затем подбираем прямую. Наклон этой прямой равен $\tau(q) + 1$. Чтобы сделать это строго, с расчетом R^2 и диагностических показателей, нам нужна полноценная реализация регрессии методом наименьших квадратов (OLS).

В нашем классе OLS для численной устойчивости используется псевдообратная матрица, вычисляемая через сингулярное разложение (SVD). Нормальные уравнения $(X'X)^{-1}X'y$ могут быть неустойчивыми при плохой обусловленности матрицы регрессоров. Подход на основе SVD раскладывает матрицу на ортогональные компоненты и обращает только ненулевые сингулярные значения. Это важно, поскольку данные для регрессии в логарифмических координатах могут охватывать несколько порядков величины.

```
//+-----+
//|                                     OLS.mqh |
//|           MMAR Library - Ordinary Least Squares Regression |
//+-----+
#property strict

#ifndef MMAR_OLS_MQH
#define MMAR_OLS_MQH

//+-----+
//| OLS class |
//+-----+
class OLS
{
private:
    bool m_const_prepend;
    matrix m_exog,
           m_pinv,
           m_cov_params,
           m_m_error,
           m_norm_cov_params;
```



```

vector m_endog,
      m_weights,
      m_singularvalues,
      m_params,
      m_tvalues,
      m_bse,
      m_const_cols,
      m_resid;
ulong m_obs,
      m_model_dof,
      m_resid_dof,
      m_kconstant,
      m_rank;
double m_aic,
      m_bic,
      m_scale,
      m_llf,
      m_sse,
      m_rsqe,
      m_centeredtss,
      m_uncenteredtss;
uint m_error;

//--- Private computation methods
ulong countconstants(void);
void scale(void);
void sse(void);
void rsqe(void);
void centeredtss(void);
void uncenteredtss(void);
void aic(void);
void bic(void);
void bse(void);
void llf(void);
void tvalues(void);
void covariance_matrix(void);

public:
      OLS(void);
      ~OLS(void);

//--- Fitting and prediction
bool Fit(vector &y_vars,matrix &x_vars);
double Predict(vector &inputs);
double Predict(double _input);

//--- Diagnostics
ulong ModelDOF(void);
ulong ResidDOF(void);
double Scale(void);
double Aic(void);
double Bic(void);
double Sse(void);
double Rsqe(void);
double Loglikelihood(void);
vector Tvalues(void);
vector Residuals(void);

```

```

vector      ModelParameters(void);
vector      Bse(void);
matrix      CovarianceMatrix(void);
};

```

Интерфейс класса устроен по образцу библиотеки Python statsmodels: метод Fit() принимает вектор зависимой переменной и матрицу независимых переменных, а диагностические показатели доступны через методы получения значений. Чтобы включить в модель свободный член, добавьте в матрицу регрессоров столбец констант — обычный прием в эконометрике. Рассмотрим основной метод подгонки параметров:

```

//+-----+
//| Fit - Perform OLS regression via SVD pseudo-inverse |
//+-----+
bool OLS::Fit(vector &y_vars,matrix &x_vars)
{
    m_endog = y_vars;
    m_exog  = x_vars;
    m_kconstant = countconstants();
    m_error = 0;
    m_model_dof = m_resid_dof = m_rank = m_obs= 0;
    m_aic = m_bic= m_llf=0;
    m_obs=m_exog.Rows();
    m_weights.Resize(m_obs);
    m_weights.Fill(1.0);
    m_rank = m_exog.Rank();
    m_model_dof = m_rank - m_kconstant;
    m_resid_dof = m_obs - m_rank;

    if(y_vars.Size()!=x_vars.Rows())
    {
        Print(__FUNCTION__," ",__LINE__," Error Invalid inputs Rows in x_vars not equal t
        m_error = 1;
        return false;
    }

    matrix U,V;
    ::ResetLastError();

    if(!m_exog.SVD(U,V,m_singularvalues))
    {
        Print(__FUNCTION__," ",__LINE__," SVD operation failed : ",::GetLastError());
        m_error = 1;
        return false;
    }

    m_pinv = m_exog.PInv();
    matrix pinv_t = m_pinv.Transpose();
    m_norm_cov_params = m_pinv.MatMul(pinv_t);

    matrix diag;
    if(!diag.Diag(m_singularvalues))
    {
        Print(__FUNCTION__," ",__LINE__," Diag operation failed : ",::GetLastError());
        m_error = 1;
        return false;
    }
}

```

```

m_rank = diag.Rank();
m_params = m_pinv.MatMul(m_endog);
m_model_dof = m_rank - m_kconstant;
m_resid_dof = m_obs - m_rank;
m_resid = m_endog - m_exog.MatMul(m_params);
scale();
covariance_matrix();
bse();
sse();
llf();
centeredtss();
uncenteredtss();
rsqe();
aic();
bic();
tvalues();

return true;
}

```

Метод разлагает матрицу регрессоров с помощью SVD и вычисляет псевдообратную матрицу Мура — Пенроуза. Оценки параметров получаются умножением псевдообратной матрицы на вектор зависимой переменной: $\text{params} = \text{pinv}(X) * y$. Затем метод оценивает коэффициенты и рассчитывает диагностические показатели: масштаб, ковариационную матрицу, BSE, SSE, логарифм правдоподобия, TSS, R^2 , AIC/BIC и t-значения. Для анализа разбиений ключевой диагностический показатель — R^2 . R^2 показывает, насколько хорошо прямая описывает зависимость в логарифмических координатах при каждом порядке момента q . Именно линейность такой зависимости лежит в основе проверки масштабирования.

R^2 рассчитывается по стандартной формуле:

```

//+-----+
//| R-squared |
//+-----+
void OLS::rsqe(void)
{
    m_rsqe = (m_kconstant)? 1 - m_sse/m_centeredtss : 1 - m_sse/m_uncenteredtss;
}

```

Поскольку мы всегда включаем в матрицу регрессоров столбец констант, R^2 равен единице минус отношение суммы квадратов остатков к централизованной общей сумме квадратов. Так определяется доля дисперсии зависимой переменной, объясняемая регрессией. Значения R^2 , близкие к 1,0, указывают на выраженное линейное масштабирование. Если R^2 ниже порога 0,60 по умолчанию, при данном q предположение о степенной зависимости не подтверждается.

Оценка показателя Херста

Показатель Херста H — важнейший параметр модели MMAR. Он определяет структуру долгой памяти компонента фракционного броуновского движения. При $H = 0,5$ памяти нет — получаем стандартное броуновское движение; при $H > 0,5$ наблюдается персистентность (тренды продолжают), а при $H < 0,5$ — антиперсистентность (тренды разворачиваются). В библиотеке есть два метода оценки: основной — по пересечению функцией $\tau(q)$ нулевого уровня, резервный — по обобщенному показателю Херста (GHE).

Метод GHE находится в отдельном заголовочном файле и оценивает $H(q)$ по структурным функциям q -го порядка. Сначала вычисляются разности ценового ряда при нескольких лагах. Затем в логарифмических координатах подбирается зависимость структурной функции от лага и определяется ее наклон. Для реализации нужны вспомогательные функции инициализации вектора и вычисления разностей с лагом:

```
#ifndef MMAR_HURST_GHE_MQH
#define MMAR_HURST_GHE_MQH

//+-----+
//| Vector arange initialization |
//+-----+
template<typename T>
void mmar_arange(vector<T> &vec, T value = 0.0, T step = 1.0)
{
    for(ulong i = 0; i < vec.Size(); i++, value += step)
        vec[i] = value;
}

//+-----+
//| Lagged difference helper |
//+-----+
bool mmar_diff_array(uint st, double &in[], vector &out, vector &other)
{
    ulong nsize = (MathMod(in.Size(), st) == 0) ? int(in.Size() / st) - 1 : int(in.Size() / st);

    if(!out.Resize(nsize) || !other.Resize(nsize + 1))
    {
        Print("resize error ", GetLastError());
        return false;
    }

    uint i, tt = 0;
    for(i = st; i < in.Size(); i += st, tt++)
    {
        other[tt] = in[i - st];
        out[tt] = in[i] - in[i - st];
    }

    other[tt] = in[i - st];

    return true;
}
```

Функция `mmar_arange` заполняет вектор значениями с равным шагом, подобно `arange` в NumPy. Функция `mmar_diff_array` вычисляет разности с шагом j : $X[j]-X[0]$, $X[2j]-X[j]$, $X[3j]-X[2j]$ и т.д. Полученный ряд используется для расчета структурной функции при каждом масштабе лага. В файле также есть вспомогательная функция `mmar_vecToArray` для преобразования вектора в массив типа `double` и перегрузка `general_hurst`, принимающая вектор и использующая эту функцию. Поэтому другие модули могут передавать оценщику GHE массив или вектор — в зависимости от того, что удобнее.

Основная функция GHE перебирает масштабы лага от нижней границы до верхней и на каждом вычисляет отношение q -го момента разностей после удаления тренда к q -му моменту уровней после удаления тренда. Наклон зависимости этой величины от масштаба в логарифмических координатах дает оценку $H(q)/q$:



```
//+-----+
//| Generalized Hurst exponent H(q) - array input |
//+-----+
double general_hurst(double &data[], int q, int lower = 2, int upper = 50)
{
    if(data.Size() < 100)
    {
        Print("data array is of insufficient length");
        return EMPTY_VALUE;
    }

    if(lower >= upper || lower < 2 || upper > int(floor(0.5 * data.Size())))
    {
        Print("Invalid input for lower and/or upper");
        return EMPTY_VALUE;
    }

    uint len = data.Size();

    int k = 0;

    matrix H;
    if(!H.Resize(ulong(upper - lower), 1))
    {
        Print(__LINE__, " ", __FUNCTION__, " ", GetLastError());
        return EMPTY_VALUE;
    }

    ///--- Main loop over lag scales
    for(int i = lower; i < upper; i++)
    {
        vector x_vector(ulong(i), mmr_arange<double>, 1.0, 1.0);

        matrix mcord;
        if(!mcord.Resize(ulong(i), 1))
        {
            Print(__LINE__, " ", __FUNCTION__, " ", GetLastError());
            return EMPTY_VALUE;
        }

        mcord.Fill(0.0);

        for(int j = 1; j < i + 1; j++)
        {
            vector dv, Y;
            if(!mmr_diff_array(j, data, dv, Y))
                return EMPTY_VALUE;

            double N = double(Y.Size());

            vector X(ulong(N), mmr_arange<double>, 1.0, 1.0);

            double mx = X.Sum() / N;
            vector XP = MathPow(X, 2.0);
            double SSxx = XP.Sum() - N * pow(mx, 2.0);
            double my = Y.Sum() / N;
            vector XY = X * Y;
```

```

double SSxy = XY.Sum() - N * mx * my;
double cc1 = SSxy / SSxx;
double cc2 = my - cc1 * mx;

vector ddVd = dv - cc1;
vector VVVd = Y - cc1 * X - cc2;
vector PddVd = MathAbs(ddVd);
PddVd = pow(PddVd, q);
vector PVVVd = MathAbs(VVVd);
PVVVd = pow(PVVVd, q);

mccord[j - 1][0] = PddVd.Mean() / PVVVd.Mean();
}

vector Px_vector = MathLog10(x_vector);

double mx = Px_vector.Mean();
vector Squ = MathPow(Px_vector, 2.0);
double SSxx = Squ.Sum() - i * pow(mx, 2.0);
matrix lmcord = log10(mccord);
double my = lmcord.Mean();
vector pt = Px_vector * lmcord.Col(0);
double SSxy = pt.Sum() - i * mx * my;

H[k][0] = SSxy / SSxx;

k++;
}

return H.Mean() / double(q);
}

```

Алгоритм выполняется в три вложенных этапа. Внешний цикл перебирает размеры окна лага от нижнего предела до верхнего. Для каждого размера окна i внутренний цикл вычисляет структурную функцию при лагах от 1 до i . При каждом лаге j вычисляются разности данных, затем простой линейной регрессией удаляется тренд, в том числе возможный дрейф. Сохраняется отношение q -й степени разностей после удаления тренда к q -й степени уровней после удаления тренда. После завершения внутреннего цикла наклон зависимости отношения от индекса лага в логарифмических координатах дает одну оценку $H(q)$. Итоговый результат — среднее по всем размерам окон, деленное на q .

Этот оценщик используется как резервный метод. Основной метод оценки H в библиотеке — поиск точки пересечения $\tau(q)$ с нулем. Находим порядок момента q^* , при котором $\tau(q^*) = 0$, затем вычисляем $H = 1/q^*$. Теоретически этот подход проще обосновать: он использует функцию разбиения, которая уже вычисляется. Резервный оценщик GHE используется, только если пересечение нуля дает неправдоподобное значение H вне диапазона 0,1-0,9. Причиной могут быть шум или недостаток данных, из-за которых $\tau(q)$ не пересекает ноль достаточно четко.

Анализ разбиений — основной вычислительный механизм

Подготовив вспомогательные модули, переходим к центральному компоненту статьи — CPartitionAnalysis. Этот класс объединяет весь пайплайн мультифрактального анализа функции разбиения. Класс получает массив логарифмических доходностей и вычисляет функцию разбиения $S_q(dt)$ по сетке порядков момента q и масштабов dt . Затем по регрессиям в логарифмических координатах определяет $\tau(q)$, оценивает показатель Херста и проводит три независимые проверки мультифрактальности данных.

Структура и конфигурация класса

Работа с классом состоит из двух шагов: инициализировать его данными, затем выполнить анализ. Для всех параметров конфигурации заданы значения по умолчанию; перед вызовом Analyze() их можно изменить:

```
//+-----+
//| CPartitionAnalysis class |
//+-----+
class CPartitionAnalysis
{
private:
    ///--- Input data
    double      m_returns[];
    int         m_n_returns;

    ///--- Configuration
    int         m_delta_t_min;
    int         m_delta_t_max;
    double      m_delta_t_spacing;
    double      m_q_min;
    double      m_q_max;
    double      m_q_step;
    double      m_min_r_squared;

    ///--- Results
    double      m_q_values[];
    int         m_n_q;
    double      m_tau_q[];
    double      m_slopes[];
    double      m_r_squared[];
    double      m_H;
    double      m_sample_volatility;
    int         m_multifractal_score;
    bool        m_is_multifractal;
    bool        m_analyzed;

    ///--- Internal computation
    double      CalculatePartitionFunction(double q, int delta_t);
    void        GenerateDeltaTValues(int &dt_values[], int &n_dt);
    void        GenerateQValues(void);
    bool        FitPartitionPlots(const int &dt_values[], int n_dt);
    double      EstimateHurst(void);
    double      EstimateHurstFromTauQ(void);
    int         CheckMultifractality(void);
    double      ComputeSampleVolatility(void);

public:
    CPartitionAnalysis(void);
    ~CPartitionAnalysis(void);

    ///--- Configuration
    void        SetPartitionConfig(int dt_min, int dt_max, double spacing);
    void        SetMomentConfig(double q_min, double q_max, double q_step);
    void        SetMinRSquared(double min_r2) { m_min_r_squared = min_r2; }

    ///--- Main pipeline
```

```

bool            Init(const double &returns[], int n_returns);
bool            Analyze(void);

//--- Results access
double          GetH(void)                { return m_H; }
double          GetSampleVolatility(void)  { return m_sample_volatility; }
void            GetTauQ(double &tau[], double &q_vals[]);
double          GetMeanRSquared(void);
bool            IsMultifractal(void)       { return m_is_multifractal; }
int             GetMultifractalScore(void) { return m_multifractal_score; }
int            GetNumQ(void)              { return m_n_q; }
};

```

Параметры конфигурации делятся на две группы. Параметры анализа разбиений задают сетку временных масштабов: `dt_min` и `dt_max` определяют диапазон, а `spacing` — плотность логарифмического размещения точек внутри него. Параметры моментов задают сетку `q`: значения перебираются от `q_min` до `q_max` с фиксированным шагом. По умолчанию `q` меняется от 0,01 до 30,0 с шагом 0,5. Получается 61 порядок момента, для каждого из которых нужна отдельная регрессия в логарифмических координатах. Сетка должна быть достаточно подробной, чтобы уловить кривизну $\tau(q)$, но не чрезмерно плотной, иначе вычисления станут слишком затратными.

Генерация временных масштабов

Значения временного масштаба располагаются по логарифмической, а не линейной сетке. Это важно, поскольку закономерности масштабирования проявляются на разных порядках величины, а не при одинаковых прибавках к масштабу. Линейная сетка от 1 до 150 дала бы 150 точек, причем большая их часть пришлась бы на малые масштабы. При логарифмическом размещении на каждый десятикратный диапазон масштабов приходится примерно одинаковое число точек:

```

//+-----+
//| GenerateDeltaTValues - Build log-spaced time scale array |
//+-----+
void CPartitionAnalysis::GenerateDeltaTValues(int &dt_values[], int &n_dt)
{
    int temp[];
    ArrayResize(temp, 200);
    int count = 0;

    double current = (double)m_delta_t_min;
    while(current <= (double)m_delta_t_max)
    {
        int dt = (int)MathRound(current);
        if(dt < 1)
            dt = 1;
        if(dt > m_n_returns / 2)
            break;

        bool duplicate = false;
        for(int i = 0; i < count; i++)
        {
            if(temp[i] == dt)
            {
                duplicate = true;
                break;
            }
        }

        dt_values[count] = dt;
        count++;
    }
}

```

```

        if(!duplicate)
        {
            if(count >= ArraySize(temp))
                ArrayResize(temp, count + 100);
            temp[count++] = dt;
        }

        current *= m_delta_t_spacing;
    }

    for(int i = 0; i < count - 1; i++)
        for(int j = i + 1; j < count; j++)
            if(temp[i] > temp[j])
            {
                int swap = temp[i];
                temp[i] = temp[j];
                temp[j] = swap;
            }

    n_dt = count;
    ArrayResize(dt_values, count);
    ArrayCopy(dt_values, temp, 0, 0, count);
}

```

Начав с dt_{min} (1), на каждом шаге мы умножаем текущее значение на коэффициент $spacing$ (1,1). Получается геометрическая последовательность: 1; 1,1; 1,2; 1,3; 1,5; 1,6; 1,8; 2; 2,2; ... вплоть до dt_{max} . Поскольку масштаб измеряется целым числом баров, мы округляем значения и удаляем повторы. Условие $dt > m_n_returns / 2$ не допускает окон длиннее половины ряда: иначе неперекрывающихся блоков останется слишком мало для осмысленной оценки функции разбиения. Полученный массив сортируется по возрастанию, чтобы регрессия в логарифмических координатах соответствовала последовательности масштабов.

Вычисление функции разбиения

В основе мультифрактального анализа лежит функция разбиения $S_q(dt)$. При заданных q и dt функция показывает среднее значение q -й степени модуля доходности по неперекрывающимся окнам размером dt .

```

//+-----+
//| CalculatePartitionFunction - S_q(dt) for given q and time scale |
//+-----+
double CPartitionAnalysis::CalculatePartitionFunction(double q, int delta_t)
{
    int N = m_n_returns / delta_t;
    if(N <= 0)
        return -1.0;

    double sum_sq = 0.0;
    for(int i = 0; i < N; i++)
    {
        double interval_return = 0.0;
        int start = i * delta_t;
        for(int j = 0; j < delta_t; j++)
            interval_return += m_returns[start + j];

        double abs_r = MathAbs(interval_return);
        if(abs_r < 1e-300)

```

```

        abs_r = 1e-300;
        sum_sq += MathPow(abs_r, q);
    }

    return sum_sq / N;
}

```

Функция делит ряд доходностей на N неперекрывающихся блоков размером Δt . Для каждого блока суммируются логарифмические доходности. Затем берется модуль полученной доходности, возводится в степень q , а результат добавляется к общей сумме. Итоговое значение — среднее по всем блокам. Нижний порог $1e-300$ предотвращает вычисление логарифма нуля при последующей регрессии в логарифмических координатах. Такая защита нужна в периоды затишья, когда несколько последовательных доходностей могут быть ровно нулевыми, особенно на старших таймфреймах или неликвидных инструментах.

Главный вывод состоит в том, что при наличии масштабирования в данных $S_q(\Delta t)$ подчиняется степенному закону: $S_q(\Delta t)$ пропорциональна $\Delta t^{(\tau(q)+1)}$. Логарифмирование превращает зависимость в линейную: $\log(S_q) = (\tau(q)+1) * \log(\Delta t) + \text{константа}$. Наклон прямой на графике в логарифмических координатах равен $\tau(q) + 1$.

Анимация на рисунке 2 показывает весь процесс: доходности объединяются в блоки, функция разбиения вычисляется при разных масштабах, а наклоны прямых в логарифмических координатах выявляют различия масштабирования для разных q :

Made By MMQ

Рис. 2. Масштабирование функции разбиения — различия наклонов при разных q указывают на мультифрактальное поведение

Подгонка прямых на графиках в логарифмических координатах

Для каждого q вычисляем $S_q(\Delta t)$ при всех масштабах Δt , берем десятичные логарифмы $S_q(\Delta t)$ и Δt , затем выполняем OLS-регрессию:

```

//+-----+
//| FitPartitionPlots - Log-log regression for each q value          |
//+-----+
bool CPartitionAnalysis::FitPartitionPlots(const int &dt_values[], int n_dt)
{

```

```

ArrayResize(m_tau_q, m_n_q);
ArrayResize(m_slopes, m_n_q);
ArrayResize(m_r_squared, m_n_q);

int valid_q = 0;

for(int qi = 0; qi < m_n_q; qi++)
{
    double q = m_q_values[qi];

    double log_dt[];
    double log_sq[];
    ArrayResize(log_dt, n_dt);
    ArrayResize(log_sq, n_dt);
    int n_valid = 0;

    for(int di = 0; di < n_dt; di++)
    {
        double s_q = CalculatePartitionFunction(q, dt_values[di]);
        if(s_q > 0.0)
        {
            log_dt[n_valid] = MathLog10((double)dt_values[di]);
            log_sq[n_valid] = MathLog10(s_q);
            n_valid++;
        }
    }

    if(n_valid < 3)
    {
        m_slopes[qi] = 0.0;
        m_tau_q[qi] = -1.0;
        m_r_squared[qi] = 0.0;
        continue;
    }

    vector y_vec;
    y_vec.Resize(n_valid);
    matrix x_mat;
    x_mat.Resize(n_valid, 2);

    for(int i = 0; i < n_valid; i++)
    {
        y_vec[i] = log_sq[i];
        x_mat[i][0] = log_dt[i];
        x_mat[i][1] = 1.0;
    }

    OLS ols;
    if(!ols.Fit(y_vec, x_mat))
    {
        m_slopes[qi] = 0.0;
        m_tau_q[qi] = -1.0;
        m_r_squared[qi] = 0.0;
        continue;
    }

    vector params = ols.ModelParameters();

```

```

        m_slopes[qi] = params[0];
        m_tau_q[qi] = params[0] - 1.0;
        m_r_squared[qi] = ols.Rsqe();
        valid_q++;
    }

    if(valid_q < 3)
    {
        Print("PartitionAnalysis::FitPartitionPlots - only ", valid_q, " valid q values")
        return false;
    }

    return true;
}

```

Для каждого q строим матрицу регрессоров из двух столбцов: предиктор $\log_{10}(dt)$ и столбец единиц для свободного члена. Зависимая переменная — $\log_{10}(S_q)$. После подгонки значение $params[0]$ равно наклону прямой, то есть $\tau(q) + 1$. Вычитаем 1 и получаем $\tau(q)$. Значение R^2 показывает, насколько хорошо степенная зависимость описывает данные при данном q . Если допустимых точек данных меньше трех — такое возможно при крайних q из-за переполнения или потери значимости очень малых чисел, — мы пропускаем это q , а не оцениваем регрессию по недостаточным данным.

Оценка показателя Херста по $\tau(q)$

Функция масштабирования $\tau(q)$ пересекает ноль при $q^* = 1/H$. Значит, чтобы оценить H , достаточно найти соответствующее q^* и взять обратное значение. Используется линейная интерполяция между двумя соседними значениями q по разные стороны от нуля функции $\tau(q)$:

```

//+-----+
//| EstimateHurstFromTauQ - Find q* where tau(q*)=0, then H=1/q*      |
//+-----+
double CPartitionAnalysis::EstimateHurstFromTauQ(void)
{
    int idx_below = -1;
    int idx_above = -1;

    for(int i = 0; i < m_n_q - 1; i++)
    {
        if(m_tau_q[i] * m_tau_q[i + 1] <= 0.0)
        {
            idx_below = i;
            idx_above = i + 1;
            break;
        }
    }

    if(idx_below < 0)
    {
        double min_abs = DBL_MAX;
        int min_idx = 0;
        for(int i = 0; i < m_n_q; i++)
        {
            if(MathAbs(m_tau_q[i]) < min_abs)
            {
                min_abs = MathAbs(m_tau_q[i]);
                min_idx = i;
            }
        }
    }
}

```

```

    }
    double q_zero = m_q_values[min_idx];
    if(q_zero <= 0.0)
        return 0.5;
    return MathMax(0.1, MathMin(0.9, 1.0 / q_zero));
}

double q1 = m_q_values[idx_below];
double q2 = m_q_values[idx_above];
double t1 = m_tau_q[idx_below];
double t2 = m_tau_q[idx_above];

double q_zero;
if(MathAbs(t2 - t1) > 1e-15)
    q_zero = q1 + (0.0 - t1) * (q2 - q1) / (t2 - t1);
else
    q_zero = (q1 + q2) / 2.0;

if(q_zero <= 0.0)
    return 0.5;
return MathMax(0.1, MathMin(0.9, 1.0 / q_zero));
}

```

Условие $\tau[i] * \tau[i+1] \leq 0$ находит соседние значения q , между которыми τ меняет знак с отрицательного на положительный или наоборот. Линейная интерполяция между ними дает значение q^* , при котором $\tau(q^*) = 0$. Тогда $H = 1/q^*$. Результат ограничивается диапазоном от 0,1 до 0,9: значения за его пределами неправдоподобны для финансовых данных и, вероятно, указывают на численный артефакт. Если функция вообще не пересекает ноль, например при очень слабом масштабировании данных, резервный метод берет ближайшее к нулю значение q как приближение.

Метод EstimateHurst() сначала оценивает H по $\tau(q)$. Если результат неправдоподобен, используется резервный оценщик GHE из предыдущего раздела.

Решение о наличии мультифрактальности

Получить $\tau(q)$ — лишь половина дела. Нужно также установить, масштабирование мультифрактально (кривая нелинейна) или монофрактально (кривая прямая, и все моменты масштабируются одинаково). В библиотеке используются три независимых диагностических теста. Каждый дает от 0 до 3 баллов, поэтому общая оценка достигает 9:

```

//+-----+
//| CheckMultifractality - Score 0-9 based on three diagnostics |
//+-----+
int CPartitionAnalysis::CheckMultifractality(void)
{
    if(m_n_q < 5)
        return 0;

    //--- Test 1: Concavity of tau(q)
    int n_concave = 0;
    int n_d2 = m_n_q - 2;
    for(int i = 0; i < n_d2; i++)
    {
        double d2 = m_tau_q[i + 2] - 2.0 * m_tau_q[i + 1] + m_tau_q[i];
        if(d2 < 0.0)
            n_concave++;
    }
    double concave_pct = 100.0 * n_concave / n_d2;

```

```

int concavity_score;
if(concave_pct >= 80.0)
    concavity_score = 3;
else
    if(concave_pct >= 60.0)
        concavity_score = 2;
    else
        if(concave_pct >= 40.0)
            concavity_score = 1;
        else
            concavity_score = 0;

//--- Test 2: Linearity (high R^2 → monofractal)
double sum_q = 0.0, sum_t = 0.0, sum_qq = 0.0, sum_qt = 0.0;
int n_pos = 0;
for(int i = 0; i < m_n_q; i++)
{
    if(m_q_values[i] > 0.0)
    {
        sum_q += m_q_values[i];
        sum_t += m_tau_q[i];
        sum_qq += m_q_values[i] * m_q_values[i];
        sum_qt += m_q_values[i] * m_tau_q[i];
        n_pos++;
    }
}

double mean_q = sum_q / n_pos;
double mean_t = sum_t / n_pos;
double ss_qq = sum_qq - n_pos * mean_q * mean_q;
double ss_qt = sum_qt - n_pos * mean_q * mean_t;
double slope_lin = ss_qt / ss_qq;
double intercept_lin = mean_t - slope_lin * mean_q;

double ss_res = 0.0, ss_tot = 0.0;
for(int i = 0; i < m_n_q; i++)
{
    if(m_q_values[i] > 0.0)
    {
        double fitted = slope_lin * m_q_values[i] + intercept_lin;
        ss_res += (m_tau_q[i] - fitted) * (m_tau_q[i] - fitted);
        ss_tot += (m_tau_q[i] - mean_t) * (m_tau_q[i] - mean_t);
    }
}

double r2_linear = (ss_tot > 0.0) ? 1.0 - ss_res / ss_tot : 1.0;
double rmse_lin = Math.Sqrt(ss_res / n_pos);

int linearity_score;
if(r2_linear > 0.99 && rmse_lin < 0.05)
    linearity_score = 0;
else
    if(r2_linear > 0.95)
        linearity_score = 1;
    else
        if(r2_linear > 0.90)
            linearity_score = 2;

```

```

        else
            linearity_score = 3;

//--- Test 3: Variation in dtau/dq slopes
double d_tau_min = DBL_MAX, d_tau_max = -DBL_MAX;
for(int i = 0; i < m_n_q - 1; i++)
{
    double d = m_tau_q[i + 1] - m_tau_q[i];
    if(d < d_tau_min)
        d_tau_min = d;
    if(d > d_tau_max)
        d_tau_max = d;
}
double slope_range = d_tau_max - d_tau_min;

int variation_score;
if(slope_range > 0.5)
    variation_score = 3;
else
    if(slope_range > 0.3)
        variation_score = 2;
    else
        if(slope_range > 0.1)
            variation_score = 1;
        else
            variation_score = 0;

int total = concavity_score + linearity_score + variation_score;

PrintFormat("Multifractality check: concavity=%d linearity=%d variation=%d total=%d",
            concavity_score, linearity_score, variation_score, total);

return total;
}

```

Тест 1 — вогнутость: кривая $\tau(q)$ для мультифрактального процесса должна быть вогнутой, то есть изгибаться вниз. Мы вычисляем вторую конечную разность в каждой внутренней точке. Если $d2 = \tau[i+2] - 2\tau[i+1] + \tau[i]$ отрицательно, кривая в этой точке вогнута. Оценка 3 ставится, если вогнутость наблюдается не менее чем в 80% точек. У строго линейной $\tau(q)$ величина $d2$ во всех точках близка к нулю; оценка — 0.

Тест 2 — линейность: это проверка, обратная первой. Мы аппроксимируем кривую $\tau(q)$ прямой и оцениваем качество подгонки. Если R^2 выше 0,99, а RMSE ниже 0,05, кривая практически линейна и данные монофрактальны (оценка 0). Совместное условие помогает избежать ложноотрицательного результата: высокое R^2 возможно и при небольшой систематической кривизне $\tau(q)$. RMSE при этом выявляет случаи, когда остатки остаются существенными. При более низком R^2 кривая заметно отклоняется от прямой, что указывает на мультифрактальную структуру (оценка до 3).

Тест 3 — изменение наклона: у мультифрактального процесса локальный наклон $d\tau/dq$ меняется в диапазоне q . При малых q локальный наклон отражает поведение крупных колебаний, а при больших q — мелких. Если наклоны существенно различаются и размах $> 0,5$, тест получает оценку 3. У монофрактального процесса наклон почти постоянен на всем диапазоне.

При суммарной оценке не ниже 5 из 9 данные признаются мультифрактальными. При разработке порог откалибровали на известных мультифрактальных и монофрактальных процессах. Порог намеренно консервативен: лучше упустить часть не слишком мультифрактальных данных, чем ошибочно признать случайный шум мультифрактальным.

Рисунок 3 показывает различие между монофрактальной и мультифрактальной кривыми $\tau(q)$ и то, как его выявляют три диагностических теста:

Made By MMQ

Рис. 3. Монофрактальная и мультифрактальная кривые $\tau(q)$ — прямая соответствует монофрактальности, вогнутая кривая соответствует мультифрактальности

Выполнение анализа на реальных данных

Подготовив все модули, мы можем написать тестовый скрипт: он загрузит рыночные данные из терминала MetaTrader и выполнит весь пайплайн анализа разбиений. Этот скрипт впервые позволит проверить всю реализацию на языке MQL5 от входных данных до результата:

```
//+-----+
//|                                     MMAR Partition Test |
//|                                     Copyright 2026, MMQ - Muhammad Minhas Qamar |
//|                                     https://www.mql5.com/ru/articles/21299 |
//+-----+
#property copyright "Copyright 2026, MMQ - Muhammad Minhas Qamar"
#property link      "https://www.mql5.com/ru/articles/21299"
#property version   "1.00"
#property strict
#property script_show_inputs

#include <MMAR\PartitionAnalysis.mqh>

input int      InpBars      = 50000;
input double   InpQMin     = 0.01;
input double   InpQMax     = 30.0;
input double   InpQStep    = 0.5;
input int      InpDtMin    = 1;
input int      InpDtMax    = 150;
input double   InpDtSpacing = 1.1;
input double   InpMinR2    = 0.60;
```

```
//+-----+
//| Script program start function |
//+-----+
void OnStart()
{
    PrintFormat("=== MMAR Partition Analysis Test ===");
    PrintFormat("Symbol: %s | Period: %s", _Symbol, EnumToString(_Period));

    //-- Load close prices and compute log returns
    double close[];
    int copied = CopyClose(_Symbol, _Period, 0, InpBars, close);
    if(copied < 1000)
    {
        PrintFormat("Not enough bars: %d (need 1000+)", copied);
        return;
    }
    PrintFormat("Loaded %d bars", copied);

    double returns[];
    int n_returns = copied - 1;
    ArrayResize(returns, n_returns);
    for(int i = 0; i < n_returns; i++)
        returns[i] = MathLog(close[i + 1] / close[i]);

    //-- Run partition analysis
    CPartitionAnalysis partition;
    partition.SetPartitionConfig(InpDtMin, InpDtMax, InpDtSpacing);
    partition.SetMomentConfig(InpQMin, InpQMax, InpQStep);
    partition.SetMinRSquared(InpMinR2);

    if(!partition.Init(returns, n_returns))
    {
        Print("Init failed");
        return;
    }

    uint start = GetTickCount();
    if(!partition.Analyze())
    {
        Print("Analyze failed");
        return;
    }

    uint elapsed = GetTickCount() - start;
    PrintFormat("Analysis time: %u ms", elapsed);

    //-- Print results
    PrintFormat("\n--- Results ---");
    PrintFormat("H (Hurst exponent): %.4f", partition.GetH());
    PrintFormat("Sample volatility: %.6f", partition.GetSampleVolatility());
    PrintFormat("Mean R-squared: %.4f", partition.GetMeanRSquared());
    PrintFormat("Multifractal score: %d/9", partition.GetMultifractalScore());
    PrintFormat("Is multifractal: %s", partition.IsMultifractal() ? "YES" : "NO");

    //-- Print tau(q) curve
    double tau[], q_vals[];
    partition.GetTauQ(tau, q_vals);
    int n_q = partition.GetNumQ();
}
```

```

PrintFormat("\n--- tau(q) curve (every 5th value) ---");
PrintFormat("%8s %10s", "q", "tau(q)");
for(int i = 0; i < n_q; i += 5)
    PrintFormat("%8.2f %10.4f", q_vals[i], tau[i]);

Print("\n=== Partition Analysis Test Complete ===");
}

```

Скрипт работает по простой схеме, которая повторится и в следующих статьях серии. Сначала мы загружаем исторические цены закрытия с помощью функции [CopyClose](#). Мы запрашиваем до InpBars баров — по умолчанию 50 000; для содержательного анализа необходимо не менее 1 000 баров. Затем вычисляем логарифмические доходности — натуральные логарифмы отношений соседних цен. Эти логарифмические доходности служат входными данными для всего последующего анализа. Наконец, настраиваем параметры анализа разбиений, инициализируем модуль данными доходностей, вызываем метод Analyze() и выводим результаты.

Посмотрим на результаты для EURUSD на таймфрейме M10 при InpBars = 500 000 вместо значения по умолчанию 50 000: так мы проверим пайплайн на более крупной выборке:

```

=== MMAR Partition Analysis Test ===
Symbol: EURUSD | Period: PERIOD_M10
Loaded 500000 bars
Multifractality check: concavity=3 linearity=1 variation=1 total=5/9
Analysis time: 3859 ms

--- Results ---
H (Hurst exponent): 0.4892
Sample volatility: 0.000422
Mean R-squared: 0.8867
Multifractal score: 5/9
Is multifractal: YES

--- tau(q) curve (every 5th value) ---
      q      tau(q)
0.01    -0.9930
2.51     0.1724
5.01     0.7441
7.51     1.2193
10.01    1.7181
12.51    2.2156
15.01    2.7076
17.51    3.1952
20.01    3.6796
22.51    4.1617
25.01    4.6422
27.51    5.1215

=== Partition Analysis Test Complete ===

```

Результаты достаточно однозначны. Показатель Херста 0,4892 близок к 0,5: общая структура дрейфа напоминает случайное блуждание. Это согласуется с гипотезой эффективного рынка на низких частотах. Однако оценка мультифрактальности 5/9 достигает порога, поэтому данные признаются мультифрактальными. Тест на вогнутость дал 3/3: кривая $\tau(q)$ выраженно вогнута, что подтверждает различия масштабирования при разных порядках момента. Среднее R^2 0,8867 показывает хорошую

подгонку степенной зависимости на всем диапазоне q и дает основания считать показатели масштабирования содержательными.

Кривая $\tau(q)$ показывает признаки мультифрактальности: начинается около -0,99 при $q = 0,01$, пересекает ноль вблизи $q = 2$ — это согласуется с H около 0,5, поскольку $1/H$ около 2, — и далее растет медленнее линейной функции. Для монофрактальных данных $\tau(q)$ была бы прямой с наклоном H . Именно эту заметную кривизну, особенно уменьшение приращений между соседними значениями по мере роста q , и фиксирует тест на вогнутость.

Теперь сравним с другим случаем. Тот же анализ EURUSD на таймфрейме M30 по 50 000 барам дает лишь 2/9 — ниже порога мультифрактальности. Это не ошибка в коде. Код корректно показывает, что при данном сочетании таймфрейма и размера выборки мультифрактальная структура слишком слаба для статистически значимого вывода. Анализ разбиений служит честным фильтром отбора: если данные не подходят для мультифрактального моделирования, это выяснится до затрат вычислительных ресурсов на построение каскада и моделирование методом Монте-Карло.

Это важно для практического применения. При использовании MMAR в советнике выбирайте таймфрейм и период анализа по тем данным, на которых действительно проявляется мультифрактальное масштабирование. На таймфрейме M10 большая выборка — 500 000 баров за несколько лет — проходит проверку. Данные на более коротких таймфреймах при меньшем числе баров могут проверку не пройти. Диагностические инструменты библиотеки позволяют выбрать параметры по результатам проверки данных, а не наугад.

Заключение

В этой статье мы создали первый крупный модуль библиотеки MMAR для MetaTrader 5. Начав с общих определений типов, мы создали модуль OLS-регрессии, оценщик обобщенного показателя Херста и полный механизм анализа разбиений. Получился автономный класс на языке MQL5: он принимает ряд доходностей, вычисляет характеристики мультифрактального масштабирования и определяет, подходит ли данным мультифрактальное моделирование.

- Файл MMARConstants.mqh задает единый набор обозначений: типы распределений, состояния модели и структуры параметров.
- Файл OLS.mqh реализует линейную регрессию на основе SVD с полным набором диагностических показателей; она используется для определения наклона в логарифмических координатах.
- Файл HurstGHE.mqh содержит устойчивый резервный оценщик показателя Херста на случай, если основным метод по $\tau(q)$ даст неправдоподобный результат.
- Файл PartitionAnalysis.mqh объединяет весь пайплайн: построение сетки временных масштабов, вычисление функции разбиения, регрессию в логарифмических координатах, оценку показателя Херста и проверку мультифрактальности тремя тестами.

Анализ 500 000 баров занимает менее четырех секунд, поэтому периодическая перекалибровка возможна и при реальной торговле. Еще важнее, что анализ дает объективную оценку: не все рыночные данные мультифрактальны, и библиотека корректно определяет случаи, когда допущение не выполняется.

В пятой части мы подгоним полученную здесь кривую $\tau(q)$ к четырем теоретическим распределениям, используя преобразование Лежандра и оптимизацию с ограничениями. Так мы получим параметры распределения для построения мультипликативного каскада — следующего шага к полному механизму моделирования MMAR.

Программы, представленные в этой статье, предназначены исключительно для образовательных целей. Они не предназначены для реальной торговли и не оптимизированы для работы в реальных рыночных условиях. Код помогает разобраться в мультифрактальном анализе. Прежде чем адаптировать его для реальной торговли, обязательно тщательно проверьте работу на демо-счетах.

Как получить исходный код через MQL5 Algo Forge

Все исходные файлы приложены к статье ниже. Полный репозиторий также доступен на [MQL5 Algo Forge](https://forge.mql5.io/ayantrader/MMAR) — платформе сообщества на базе Git для обмена торговыми проектами и совместной работы над ними. Algo Forge хранит историю версий локально и в облаке, поэтому вам будет доступна актуальная версия кода, в том числе обновления и исправления, внесенные после публикации статьи.

Чтобы клонировать репозиторий, откройте командную строку, PowerShell или терминал, встроенный в ваш редактор кода, и выполните команду:

```
git clone https://forge.mql5.io/ayantrader/MMAR.git
```

Для этого на вашем компьютере должен быть установлен [Git](https://git-scm.com/). Если вы работаете в Visual Studio Code, откройте встроенный терминал сочетанием клавиш Ctrl+` и выполните команду прямо в нем. Репозиторий будет клонирован в папку MMAR в текущем каталоге.

Кроме того, проект можно просматривать по адресу forge.mql5.io/ayantrader/MMAR: исходные файлы, историю коммитов и обновления можно открыть прямо в браузере. В отличие от приложенных файлов, Algo Forge дает доступ ко всей истории Git и позволяет загружать будущие обновления одной командой:

```
git pull
```

Имя файла	Описание
MQL5\Include\MMAR\MMARConstants.mqh	Общие перечисления, структуры и определения типов библиотеки MMAR
MQL5\Include\MMAR\OLS.mqh	Регрессия методом наименьших квадратов через псевдообратную матрицу на основе SVD с полным набором диагностических показателей
MQL5\Include\MMAR\HurstGHE.mqh	Оценщик обобщенного показателя Херста (резервный метод)
MQL5\Include\MMAR\PartitionAnalysis.mqh	Мультифрактальный анализ функции разбиения с диагностикой масштабирования
MQL5\Scripts\MMAR\MMAR_PartitionTest.mq5	Тестовый скрипт для демонстрации анализа разбиений на реальных рыночных данных

Следующая статья >>

Muhammad Minhas Qamar

[Пакистан](#)

[9400](#)

Developer by Profession, Trader by Hobby

Gmail: ayanminhasshayar@gmail.com

Другие статьи автора

- [За пределами GARCH \(Часть VII\): Прогнозирование волатильности методом Монте-Карло в MQL5](#)
- [За пределами GARCH \(Часть VI\): Фракционное броуновское движение и мультипликативный каскад в MQL5](#)
- [За пределами GARCH \(Часть V\): Подгонка мультифрактального спектра в MQL5](#)
- [За пределами GARCH \(Часть III\): Построение MMAR и вердикт](#)
- [За пределами GARCH \(Часть II\): Оценка фрактальной размерности рынков](#)
- [За пределами GARCH \(Часть I\): Сравнение моделей MMAR Мандельброта и GARCH Энгла](#)

[Подпишитесь](#) и узнавайте о
популярных статьях и кодах